



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

FORTRAN PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Programming Languages

TOTAL: 10 QUESTIONS

Q1 What are the primary design goals of Fortran?

Threat Model

Q6 How does Fortran implement object-oriented or functional programming?

Functionalability

Q2 How does Fortran handle type checking: static or dynamic?

Architecture

Q7 How does Fortran handle errors?

Lifecycle

Q3 How does Fortran manage memory?

Configuration

Q8 What testing tools are commonly used in Fortran?

Compliance

Q4 What is Fortran's concurrency model?

Hardening

Q9 What makes Fortran well-suited for its target domain?

Testing

Q5 How are packages and dependencies managed in Fortran?

OS / IDP

Q10 What are common performance pitfalls in Fortran?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Fortran was designed with specific priorities around

Mitigation

Fortran was designed with specific priorities around performance, safety, or developer productivity depending on its domain. These goals shape its syntax, type system, and standard library choices.

A6 Fortran supports classes and inheritance for OOP,

Observability

Fortran supports classes and inheritance for OOP, or first-class functions and immutability for FP, or both. Many modern Fortran programs blend both paradigms depending on the problem.

A2 Fortran uses its type system to catch

Primitives

Fortran uses its type system to catch errors at compile time (static) or at runtime (dynamic). This affects refactoring safety, tooling support, and the kinds of bugs that reach production.

A7 Fortran surfaces errors via exceptions, Result/Either types,

Configuration

Fortran surfaces errors via exceptions, Result/Either types, or error return values. The choice impacts whether errors are checked at compile time and how calling code is structured.

A3 Fortran uses one of three approaches: manual

Hardening

Fortran uses one of three approaches: manual allocation/deallocation, a garbage collector that periodically reclaims unreachable objects, or automatic reference counting. Each has different latency and throughput tradeoffs.

A8 Fortran has a standard or widely adopted

Governance

Fortran has a standard or widely adopted test runner and assertion library. Tests are typically organized into unit, integration, and end-to-end layers with coverage reporting built in or via plugins.

A4 Fortran supports concurrency through threads, coroutines,

Interchange

Fortran supports concurrency through threads, coroutines, async/await, or an actor model. The choice determines how I/O-bound and CPU-bound workloads are scaled and how shared state is safely accessed.

A9 Fortran excels in its domain due to

Assessment

Fortran excels in its domain due to a combination of performance characteristics, ecosystem maturity, language ergonomics, and community support that makes common tasks simple.

A5 Fortran uses a package manager and a

Federation

Fortran uses a package manager and a manifest file to declare dependencies and version constraints. A lock file pins exact versions to ensure reproducible builds across environments.

A10 Typical performance issues in Fortran include excessive

Optimization

Typical performance issues in Fortran include excessive allocations, blocking the main thread or event loop, $O(n^2)$ data structures, and missing caching. Profiling and benchmarking tools are available to diagnose them.